PRÁCTICA FINAL

Integrantes:

Juan Daniel Vivas Granada

Sebastián Andrés Ibarra Prada

Docente:

Carlos Alberto Álvarez Henao

Asignatura:

Estructura de Datos y Algoritmos

Institución:

Universidad EAFIT

Programa:

Ingeniería de sistemas

Ciudad:

Medellín, Colombia

Fecha:

08/05/2026

Introducción

En el siguiente informe se encuentra documentado todo con relación al diseño, implementación y análisis de los algoritmos de grafos propuestos en la práctica final de la materia Estructura de Datos y Algoritmos. La aplicación de estos algoritmos se basa en un grafo real de gran escala, la red vial de Pensilvania (EE. UU.), obtenido del repositorio SNAP de la Universidad de Stanford. El programa recibe como entrada el archivo roadNet-PA.txt (o una versión preprocesada de él) y ejecuta tres módulos independientes seleccionados por argumento de línea de comandos.

Los objetivos del presente trabajo son:

- Leer, parsear y cargar un dataset de grafos de escala real (~1M nodos, ~3M aristas) en estructuras eficientes implementadas en C/C++.

- Implementar Dijkstra con heap mínimo para resolver consultas de camino mínimo punto a punto (P2P-SSSP) sobre el grafo vial completo.

- Analizar propiedades estructurales básicas del grafo (grado, componentes conexas, diámetro aproximado) y relacionarlas con los valores publicados por SNAP.

- Comparar empíricamente los tiempos de ejecución de Dijkstra contra BFS no ponderado y discutir las diferencias con base en la complejidad teórica.

- Diseñar y ejecutar un conjunto de consultas reproducibles sobre el dataset, documentar los resultados e identificar patrones propios de redes viales reales.

- Construir un README claro que permita a cualquier persona reproducir todos los resultados desde cero.

Descripción del Dataset

El dataset roadNet-PA.txt corresponde a la red vial del estado de Pensilvania publicada por SNAP. Tras cargar y procesar el archivo, se obtuvieron las siguientes estadísticas estructurales:

Numero de nodos: 1090920 contra 1,088,092 del valor SNAP

Numero de aristas: 1541898 contra 1,541,898 del valor SNAP

Promedio de grado: 2.82678 contra 2.83 del valor SNAP

Grado mayor: 674502

Diametro aproximado: 592 contra 794 del valor SNAP

Numero de componentes conexas: 3034

Numero de nodos en la componente conexa principal: 1087562 contra 1,087,562 del SNAP

El número de aristas y el grado promedio coinciden exactamente con los valores publicados por SNAP, lo que confirma que el parseo fue correcto. El número de nodos presenta una diferencia menor explicada por nodos aislados que el programa cuenta como parte del vector de adyacencia aunque no tengan aristas. El diámetro aproximado de 592 es menor al publicado por SNAP (794) porque se estimó lanzando un único BFS desde el nodo de mayor grado, no desde múltiples nodos.

De los resultados se puede inferir que cada intersección conecta en promedio con menos de 3 otras intersecciones, lo que explica el grado promedio de 2.83. Por otro lado el diámetro alto refleja que Pensilvania es geográficamente extensa y para ir de un extremo al otro hay que atravesar varias intersecciones.

Diseño e implementación

- **Parseo del Dataset**

El parseo del archivo RoadNet-PA.txt se implementó en loader.cpp, como está indicado en el enunciado de la práctica, mediante la función ParsearArchivo, la cual lee el archivo líne por línea ignorando líneas vacías o que empiecen por el carácter # y extrae los dos enteros con stringstream. Las aristas se recuperan como un vector<Arista> que luego se procesa en graph.hpp para construir la lista de adyacencia.

- **Estructuras de datos**

Para representar el grafo se utilizó una lista de adyacencia implementada en un archivo graph.hpp para su posterior uso en los diferentes archivos .cpp mediante vector<vector<int>> para el grafo no ponderado y vector<vector<PesoNodo>> para el grafo ponderado donde PesoNodo es un struct que almacena el nodo vecino y el peso de la arista. La razón por la que se utiliza una lista de adyacencia en lugar de una matriz es porque el grafo es muy disperso, lo que haría que el uso de memoria se elevara en comparación a la lista de adyacencia. Para ilustrar esto de mejor forma, teniendo en cuenta la cantidad de nodos (1.088.092), este valor elevado al cuadrado y multiplicado por 4 (bytes), el espacio en memoria de la matriz es de 4.73 TB de memoria, lo cual vuelve inviable el uso de la matriz para la resolución de este problema.

Para la implementación del BFS se tomó como base el pseudocódigo visto en clase. El algoritmo recorre el grafo nivel por nivel desde un nodo origen, explorando primero todos los vecinos directos antes de avanzar a los siguientes niveles. Esto garantiza que la primera vez que se alcanza un nodo, se hace por el camino con menor número de saltos:

```
pseudo

cola C; vértices v, w; ord = 0;

Mientras queden vértices sin visitar

   v = elegir un vértice no visitado;

   marcar v;

   C.vaciar();

   C.encolar(v);

   Mientras C no sea vacía

      w = C.primero();

      ord++;

      orden(w)=ord;

      C.desencolar();

      Para cada vecino z de w

         Si z no está visitado

            marcar z;

            C.encolar(z);
```

Para Dijkstra se tomó como referencia la implementación con heap mínimo de la página GeeksforGeeks y se adaptó a lo indicado en el proyecto. El heap almacena pares (distancia, nodo), donde cada vez que encuentra un camino más corto a cierto nodo se inserta una nueva entrada al heap sin eliminar la anterior. Para reconstruir el camino completo a partir de prev[] se implementó la función reconstruirCamino, que recorre el arreglo desde el nodo destino hasta el origen

siguiendo la cadena de predecesores. Los nodos se van agregando a un vector y al final se invierte con reverse para obtener la secuencia correcta de origen a destino. Los pesos de las aristas fueron asignados aleatoriamente con rand() % 10 + 1 usando semilla fija srand(42) como solicitado en el enunciado de la práctica.

Análisis de Complejidad

Para el caso del BFS, se recorre cada nodo exactamente una vez gracias al vector visitados[], y por cada nodo recorre todas sus aristas. En total se visitan V nodos y E aristas, dando una complejidad de O(V + E). Para este grafo con ~1M nodos y ~3M aristas esto es perfectamente viable, ya que el número de operaciones es proporcional al tamaño del grafo y no a su cuadrado.

Para Dijkstra cada nodo se extrae del heap mínimo a lo mucho una vez: O(V log V), por cada arista se puede realizar una inserción en el heap: O(E log V), dando así que la complejidad total es O((V + E) log V), aún siendo menor que el cuadrado del tamaño del grafo.

Resultados de Consultas P2P

consulta	origen	destino	dist_dijkstra	saltos_bfs	nodos_dijkstra	nodos_bfs	t_dijkstra_ms	t_bfs_ms
1	1	500000	343	101	61231	54713	25	7
2	100	1000000	1529	484	1007449	1008999	398	118
3	50000	750000	1802	545	949961	925233	379	104
4	200000	800000	398	124	92234	88414	52	10
5	300000	100000	2072	666	1052769	1051816	387	113
6	1	1087562	300	95	45119	47273	16	6
7	500000	1	343	101	92699	90936	33	9
8	250000	600000	957	300	745739	758054	290	68
9	10000	900000	580	169	179581	156513	91	20
10	400000	150000	578	167	390229	352068	158	30

Lo primero que se puede observar es que el BFS es consistentemente más rápido que Dijkstra. Esto tiene sentido considerando que BFS trabaja con una cola simple de operaciones O(1), mientras que Dijkstra mantiene un heap mínimo donde cada inserción y extracción cuesta O(log V).

En cuanto a las distancias, Dijkstra reporta un valor mayor que los saltos de BFS porque miden cosas distintas, es decir, BFS cuenta el número de aristas recorridas, mientras que Dijkstra acumula la suma de los pesos. Como los pesos se asignaron aleatoriamente entre 1 y 10, una ruta con pocos saltos, pero aristas pesadas, puede tener mayor costo que una ruta más larga pero con aristas ligeras.

Respecto a los nodos explorados, Dijkstra tiende a explorar más nodos que BFS en la mayoría de las consultas. Esto ocurre porque el heap mínimo prioriza los nodos de menor distancia acumulada, lo que puede llevarlo a explorar zonas alejadas antes de encontrar el destino.

Análisis del Subgrafo Inducido

El subgrafo inducido por los caminos de Q01 y Q06 resultó en 221 nodos y 448 aristas, representando dos corredores viales que parten del mismo origen y se dirigen a destinos opuestos en Pensilvania, compartiendo algunos tramos iniciales antes de divergir.

El algoritmo de Kruskal calculó un MST con peso total de 1038, conectando los 221 nodos con las 220 aristas de menor costo posible sin formar ciclos. Finalmente, el análisis DFS confirmó que el subgrafo es un DAG, lo que es consistente con que los caminos mínimos no generan ciclos entre sí.

Herramientas utilizadas

Durante el desarrollo de la práctica se utilizaron herramientas de inteligencia artificial, principalmente Claude y ChatGPT, como apoyo para la realización de la misma. La IA se usó puntualmente para consultar conceptos teóricos sobre los algoritmos implementados, recibir retroalimentación sobre errores puntuales de compilación y verificar si el código escrito por el equipo era correcto. También se usó para explorar alternativas de implementación. Algunos prompts representativos fueron: *“"¿qué diferencia hay entre pasar un parámetro por referencia y por valor?", “¿cómo evito que el vector de adyacencia se salga de rango al insertar nodos con IDs grandes?”, “¿qué significa que un grafo sea disperso y por qué afecta la elección de estructura de datos?”.* En ningún momento se solicitó que la IA generara módulos completos, todo el código fue escrito por los integrantes del equipo como se puede verificar en el GitHub mediante los commits realizados.

Conclusiones

De la práctica se puede concluir que el estudio y análisis de los grafos es una herramienta muy importante para la comprensión de estructura aplicadas en la vida real, ya sea como objetivo la organización vial o de cualquier otro tipo, en donde las distintas formas de poder solucionar un mismo problema abren paso a varias posibilidades dependiendo del enfoque con el que se trabaje, ya sea en estructuras como listas de adyacencia o y matrices de adyacencia, entre otras estructuras usadas para la resolución de estos problemas.

Fue interesante ver cómo BFS y Dijkstra, siendo algoritmos que resuelven problemas similares, se comportan de forma tan distinta en la práctica. BFS fue hasta 5 veces más rápido en todas las consultas, pero eso no lo hace mejor, solo que simplemente resuelve un problema diferente. De esto concluimos que saber cuándo usar cada uno de los enfoques es tan importante como saber implementarlos.

Finalmente, trabajar con datos reales enseña que los resultados teóricos no siempre coinciden exactamente con los publicados, por ejemplo, pequeñas diferencias en el conteo de nodos o en la estimación del diámetro son esperables y deben saberse interpretar y justificar.

Referencias

GeeksforGeeks. (2025, 20 diciembre). *Kruskal's Minimum Spanning Tree (MST) Algorithm*. GeeksforGeeks. https://www.geeksforgeeks.org/dsa/kruskals-minimum-spanning-tree-algorithm-greedy-algo-2/

GeeksforGeeks. (2026, 21 enero). *Dijkstra's algorithm*. GeeksforGeeks. https://www.geeksforgeeks.org/dsa/dijkstras-shortest-path-algorithm-greedy-algo-7/

Leskovec, J. & Krevl, A. (2014). SNAP Datasets: Stanford Large Network Dataset Collection. http://snap.stanford.edu/data

Álvarez, C. (s. f.). *EstructuraDatosAlgoritmos1/S08_EDNoLineales_Grafos.ipynb at main · carlosalvarezh/EstructuraDatosAlgoritmos1*. GitHub. https://github.com/carlosalvarezh/EstructuraDatosAlgoritmos1/blob/main/S08_EDNoLineales_Grafos.ipynb